Polyphony: A Publishing Protocol for Independent Authors and Collective Publications

Abstract

> Writers conform less because anyone tells them to than because dissent is expensive: there are few publications to move to, and moving means abandoning the work. This paper proposes a protocol that lowers both costs. It takes the press, which turns a manuscript into a page, out of the publication's hands and runs it as open code on sealed hardware that neither author nor publication controls. First, starting a publication takes little effort. Second, an author can end an agreement alone, at once, on a public ledger; the pages are rebuilt without the work, and no one can stall that. Finally, every page shows who wrote it, that both sides consented, and that it is unchanged. What remains is trust in that hardware, and in its owners not to collude. A publication keeps its writers only while they choose to stay.

1. Introduction

Much of the media speaks in unison, and often no one has ordered it. Writers fall in line because disagreeing costs them too much. First, there are only a handful of places to publish, so a writer who falls out with one has few others to turn to. Second, leaving usually means leaving their work behind. Most writers find it safer to go along. Polyphony addresses both. First, anyone can start a publication with little effort, alone or with others. Second, writers keep their work when they leave. With more publications competing for writers, a writer who disagrees has somewhere to go, and the media has less reason to speak with one voice.

Three situations show the problem as writers and editors meet it today. Two journalists work a beat together for years: the same sources, the same enemies, a running series of investigations that readers cite by the site’s name rather than by either byline. The site belongs to one of them. Half the archive is the other’s work. Then a question arrives that splits them, and the other says so in public. Shortly after, he is gone. He says he was fired for dissent; the owner says he was let go for cause. Whichever is true, years of his work still stand on the site, under the masthead of the owner he is accusing, and whether they stay, come down, or quietly change bylines depends on the owner’s goodwill, not on anything he controls.

A group of writers runs a publication on a platform that promised to stay out of the way. It does, including when it starts carrying, and taking a cut from, writing that some of the group cannot stand beside. Half want to leave at once. Half say the audience is here, and principles don’t pay rent. The email list can be exported. The rest cannot: the paying readers, who found them through the platform’s own recommendations, and the years of links pointing at the platform’s address. Whoever leaves takes a list and starts over. Whoever stays keeps the rest. The disagreement was about the writing. The split is about the platform’s terms, which none of them set and none of them can change.

A war breaks out, or a law passes that divides a country, and an editor sets out to publish the widest range of serious views on it, side by side, under one name. The invitations go out. Some say yes at once, mostly people with little to lose, whose positions are already known. Others hesitate. They don’t yet know who else will be in it, and once their piece is in, it stays beside whatever arrives next. If one contribution later becomes unbearable, leaving means asking the editor to take their piece down, and the editor can agree, delay, argue or refuse. A writer who is refused has one lever left, which is to go public, and then the withdrawal becomes the story. So the most careful writers stay out, and the range the editor wanted narrows before the first piece appears.

In all three cases, the disagreement is settled by something other than the writing: by who owns the infrastructure. Writers and editors who publish together have two options today, and both end there. They can join a platform, whose terms can change or end without them. Or they can host their own site, which puts the site, the subscriber list, the archive and the revenue in the hands of whoever holds the credentials. Editors

lose too, because the careful writers they most want stay out. What accumulates is not a policy but a habit, and writers learn to either align or stay out. The result is a publication that speaks with one voice and can point to no instance of having enforced it.

Polyphony is a protocol, not a platform. Publishing has always had three parties: the author, the publication, and the press that turns a manuscript into something readers can see. The relationship was never equal. The author could not take a manuscript to the press directly. The publication controlled the relationship with the press, and much of its power over the author came from that. Online, the press is the server, the publishing software or the platform. Polyphony moves the press into a neutral position between the two.

Its press is open code running on sealed hardware that not even its owner can see into. It decrypts a manuscript, renders the page and signs it with its own key, and only while an agreement covers that manuscript, so neither the author nor the publication can produce a valid page alone. Nor can either side act on the other's work: either can end an agreement without the other's consent, and no one but the author can change what the author wrote. What is left is trust: in that hardware, and in the people who own it not to collude. Section 7 says how that trust can be raised.

The rest follows from that. The author's manuscripts are stored encrypted, and only the press can open them: for the author at any time, for a publication's reviewers while they consider one, and for building pages only while an agreement covers it. Readers see the finished pages, never the manuscript. The author shares a manuscript with a publication under an agreement. Each agreement is recorded on a public ledger that neither party operates, so ending one is a fact, not a request. Once the author ends their agreement, the manuscript can no longer be decrypted for the publication's pages, and the author, or anyone acting for the publication, can have the site rebuilt without it, so what readers see follows the agreements in force. No person stands in between who could delay, argue or refuse.

Because neither the author nor the publication controls the press, its signature means something. Every page it produces carries that signature, and points to the manuscript, which the author signed, and to the agreement on the ledger. Together they show who wrote the page, that author and publication agreed to publish it, and that the copy in front of the reader is unchanged since it was rendered. The author is identified by an account, which may be a pseudonym, but the account is public, and every page published under it points back to it. So a byline becomes something a reader can check, and a reputation becomes something an author builds over time and no one else can claim. That matters more as machine-written text becomes cheap and plentiful. The protocol cannot tell whether a person or a model wrote the words. It can show who stands behind them, and that, over time, is what a reader has to go on.

It is built for places where the threat to editorial independence is less the state than whoever owns the infrastructure. It is no defence against a state-level adversary: a government that goes after a journalist can compel the parties to any arrangement, this

one included. Nor does it decide the business model, or how its own machinery is governed. Payment is up to each publication: the protocol leaves room for any model, and offers rails on which revenue reaches the author directly instead of being passed on by the publication. Governance is up to each implementation of Polyphony: its governance chooses the code the press runs, and can add restrictions, censorship included. It cannot do so silently, and anyone who objects can fork the protocol and run another implementation. Polyphony makes the conditions of publishing neutral and public. It does not promise a world without censorship.

Go back, then, to the three cases.

The journalist who was let go would still be in a dispute with the site’s owner, and the protocol would not decide who was right. But his years of investigations would no longer depend on the owner’s goodwill. He could end his agreements and have the site rebuilt without them himself, and he would be free to publish them elsewhere. No one but him could change a word or a byline.

The writers whose platform began carrying writing they could not stand beside would not be on someone else’s platform at all. The name and address their readers know would be their own. They might still disagree about what to do. But if they split, each half would keep its own work, and either could form a publication of its own.

The editor gathering the widest range of views on the war, or the law, could get the careful writers to say yes. They would not need to know in advance who else would join, because if a later piece became unbearable, they could withdraw their own piece themselves, without asking the editor and without going public.

In each case, disagreement has somewhere to go. A publication that wants to keep its writers has to persuade them, and the writers who stay are there by choice. That is what polyphony means: many voices under one name, held together by argument rather than by ownership.

2. The author

Writing is local, and publishing is a separate, deliberate act. An author writes and revises on their own machine, and nothing in the protocol has to be reachable for that. The protocol defines an interface, not an application, so no single program stands between an author and their manuscripts: any number of writing tools can publish through it, and ideally the same tool works with more than one implementation.

To publish, the author signs the manuscript and encrypts it, not to any person or company, but to the press's key. After that, only the author can decrypt it at any time; the reviewers of a publication it has been offered to can decrypt it while they consider it; and the renderer can decrypt it while an agreement with that publication exists. A few details stay public, such as its tags, so that a publication can sort what it has been offered, and organise its pages by topic, before anything is decrypted. Anyone can read them, and so anyone can see that an author offered a manuscript to a publication.

The encrypted manuscript goes to distributed storage, and the author's own copy stays with them. Keeping the stored copy available is the author's responsibility, on their own storage or through a service of their choice. Every revision is signed and recorded. Sharing a manuscript is an offer, not a transfer, and the author can make the same offer to anyone else.

The author can end an agreement at any time, alone, and the ending takes effect at once. It is one entry on the ledger. From that moment the renderer can no longer decrypt the manuscript for that publication, and the author can trigger the rebuild themselves instead of waiting for the publication to do it. The publication cannot keep showing the old site under its name, because only the protocol can change what that name points to, and it points it at the rebuilt site, which no longer carries the work. Copies already read, saved or archived elsewhere cannot be recalled. What the author can stop is the publication presenting the work as part of what it publishes now.

Behind every byline is an account. The account is public and permanent. The keys that control it are neither: they stay private, and they can be replaced, so an author's history does not hang on a single secret. If every key is lost, a recovery method restores control of the account. The account may be pseudonymous; the protocol never holds a legal name, though an author is free to tell anyone they like. Pseudonymous is not anonymous: everything an account publishes stays linked to it. An account may also name a wallet, so that any payment due to the author arrives there directly, not through the publication's hands. It may likewise carry a name recorded on the ledger, so that a byline can show a name rather than an address.

Every author also keeps a public list of what they have published: their works and their agreements with publications, past and current. The list sits on the ledger, holds references and proofs rather than the manuscripts themselves, and only the author curates it. This makes the byline portable. Long after leaving a publication, an author

can still prove what they published there and when, and anyone can check it against the ledger. For the work to be read again, the author offers the manuscript to another publication.

What the protocol guarantees an author

An implementation must let every author:

1. Write and revise without the network; publishing is a separate, deliberate act.
2. Sign every manuscript and encrypt it to the press's key, so that only they, the reviewers of a publication it is offered to, and the renderer under an agreement can decrypt it.
3. End any agreement alone, at any time, with immediate effect, and trigger the rebuild of the site without the work.
4. Revise and reissue a manuscript, with every version signed and recorded.
5. Rotate keys, or recover the account through a recovery method, without losing their account, history or reputation.
6. Publish pseudonymously; the protocol never holds a legal name.
7. Be paid directly, at a wallet their account names.
8. Offer the same manuscript to any other publication; the protocol imposes no exclusivity, and any exclusive is a contract between the parties.
9. Keep a public list of their works and agreements, past and current, that only they curate and that survives leaving any publication.

3. The manuscript

A manuscript’s identity is derived from the first version the author signs, and every revision keeps it. Agreements are made over that identity, so an agreement outlives a revision.

The author signs a manuscript once. The signature covers its public details and a hash of its text that sits among them, so it proves which account authored the manuscript, and lets anyone holding the text check later that it hasn’t changed. The body is encrypted to the press’s key, and anyone can encrypt to that key; without the hash, a stranger’s text could be rendered under the author’s account. The hash is salted with a random value kept inside the encrypted body, and the press renders a body only if it matches.

The signed part names the published format the manuscript follows, so other implementations can read and verify it. Anyone holding the text can check which account authored it without asking the publication.

What the protocol guarantees about a manuscript

An implementation must make every manuscript:

1. Have an identity derived from its first signed version, which every revision keeps and over which agreements are made.
2. Carry, in its signed part, a salted hash of the body’s plaintext.
3. Name, in its signed part, the published format it follows.
4. Be verifiable by anyone holding its text, without the publication.

4. The publication

A publication is a name, a look and an editorial line: which authors it works with, and which manuscripts it publishes. The name is the address its pages are served from, and it is the publication's own, not space rented on someone else's platform. Keeping its pages available is the publication's own responsibility, as keeping manuscripts available is the author's. There are no constraints on how it looks: the publication writes its own templates and styles. It is its own thing and looks like it.

How it decides is up to the publication. It may have one holder or several, and set whatever threshold it likes for acting. Holders make the publication's decisions; editors and reviewers are the people they authorise to read and judge manuscripts. The protocol does not fix that shape; it makes it public. Every decision is an act on the ledger: trusting an author, admitting a manuscript, changing the site's design, changing the governance itself. All of them go through the same procedure, and none is silent. The holders and editors may be pseudonymous, as authors may, but the trail of their decisions is public. No decision by the publication reaches the next build of the site any other way.

A manuscript reaches the site by one of two routes. An author the publication trusts offers a manuscript, and the agreement takes effect at once: the manuscript goes straight to the renderer. Any other author's manuscript waits for review: the publication's reviewers can read it, and the publication then admits or declines it. The publication can change its stance toward an author at any time. It can decline a manuscript, on the record, but it cannot change one. An editor can propose a new version, and it becomes one only when the author accepts it and signs it. The protocol leaves room for that; whether an author can receive such proposals depends on their writing tool. That suits what Polyphony is for: signed writing, such as opinion, essays and analysis, where the byline answers for the text. It is not built for a news desk, where the masthead answers for every word.

An author can also revise a manuscript after it is published, and every revision puts it back among the manuscripts waiting for review. From an author the publication does not trust, it waits there like any new manuscript, and the last approved version stays on the site in the meantime. From a trusted author, it is approved at once and goes straight to the site, and the publication cannot stop it. The publication does see every change: each revision is recorded, and it is notified. If it does not want to carry the new version, it can end the agreement, and the manuscript comes off the site. Until it does, the new version stands under its name. A publication that does not want that risk trusts no one and reviews everything.

For an author's ending of an agreement to mean anything, the publication must not be able to keep its name pointing at an old site. So the name is kept on the ledger, and when it joins, the publication gives the protocol the sole right to change what the name points to. From then on only the renderer changes what the name points to, and only to a site it has just built from the agreements in force. The publication takes that right

back only by leaving. A publication that leaves can point its name anywhere, an old build included. That cannot be prevented, but it can be detected: anyone checking the name can see that the protocol no longer controls it, and any tool that checks a page's signature can check this too.

What the protocol guarantees a publication

An implementation must let every publication:

1. Use its own name, and design its site with its own templates and styles.
2. Choose its own governance, with one holder or several and any threshold, recorded in public.
3. Let its holders and editors act pseudonymously; the protocol never holds their legal names.
4. Decide which authors it trusts and which manuscripts it admits, and change either at any time, on the record.
5. Read any manuscript offered to it, through the reviewers it has authorised, until it declines it or the author withdraws it.
6. Decline any manuscript, and end any agreement from its side, with immediate effect.
7. Trigger a rebuild of its site at any time.
8. Keep its name pointing only at sites the renderer has built from the agreements in force.
9. Take back the right to change what its name points to by leaving the protocol.
10. See every revision of a manuscript it carries, be notified of it, and review it first unless it trusts the author.
11. Keep a record of everything it has published and every byline it has carried; an ended agreement is marked as ended, never removed.
12. Carry every manuscript exactly as its author signed it; no one, the publication included, can alter one.
13. Be paid directly, at a wallet it names.

5. The agreement

An agreement is between an author and a publication, over a single manuscript. It records who wrote the manuscript, that the publication may publish it, how any revenue divides between them, and whether, and for how long, a reader must pass the reader gate to read it (sections 6 and 8). The split is public, because a reader who pays needs to know where the money goes; section 8, on monetization, describes how it is paid. Both parties enter it: the author by offering the manuscript, the publication by admitting it, or in advance, by trusting the author. Withdrawing that trust later affects only future offers; agreements already made stand.

What a publication publishes is exactly the set of agreements in force, each showing the latest version the publication has approved; from a trusted author, that is always the latest. There is no separate list: a manuscript is published because an agreement covers it, and stops being published when that agreement ends, every version with it. Admitting a manuscript is all it takes to put it on the site; the next build carries it.

Agreements are recorded on the ledger, where neither party can rewrite them. Nor can anyone but the implementation's governance, by publishing new code, change the rules an agreement runs under, meaning how it is entered and ended, and who may decrypt under it. They are set in the manuscript's gates when it is encrypted (section 6). An agreement transfers nothing: the work always remains the author's, and the publication's permission to publish it lasts only as long as the agreement.

What the protocol guarantees about an agreement

An implementation must make every agreement:

1. Cover a single manuscript, and show the latest version the publication has approved.
2. Record the author, the publication's permission to publish, how any revenue divides, and whether, and for how long, a reader must pass the reader gate to read it.
3. Need the consent of both parties, the publication's given per manuscript or in advance by trusting the author.
4. End when either party ends it, alone, from the moment the ending is recorded on the ledger.
5. Be the only grounds on which the renderer decrypts a manuscript for a publication.
6. Sit on the ledger, where neither party can rewrite it, and stay there when it ends, marked as ended.
7. Run under rules that no one but the implementation's governance, by publishing new code, can change once the manuscript is encrypted.

6. Gates

A manuscript is encrypted to the press's key. Who can decrypt it, and when, is settled by gates: conditions fixed in advance, which no one can change afterwards. What a gate reads from the ledger, such as whether an agreement is in force, can change; the gate itself cannot. Each gate is checked at the moment someone asks, and must pass before anything is decrypted. A gate decides only whether a manuscript can be decrypted, not what is done with it afterwards. Where the gates run, and what has to be trusted about them, is the subject of section 7.

Two gates open a manuscript to people. The author can decrypt their own manuscript at any time. The reviewers of a publication can decrypt a manuscript offered to it, until the publication declines it or the author withdraws it. What they have read by then stays read; they just cannot decrypt it again.

Two gates open it to the renderer, the part of the press that builds a publication's pages. The renderer decrypts a manuscript only while an agreement covering it is in force, and only the version the publication has approved. And it builds and publishes a site only when asked by someone with the right to ask: the author of a manuscript the site carries or has carried, a holder of the publication, or someone the holders have authorised. A request cannot put anything on the site, because the renderer builds only from the agreements in force.

The last gate is for readers, and only when an agreement includes monetization (section 8). Whether a manuscript sits behind it, and for how long, is agreed by both parties along with the split, and holds for as long as the agreement does; what the gate asks of a reader, such as a payment or a membership, is the publication's choice (section 8). Neither party can change what they agreed alone: changing it means ending the agreement, which takes the manuscript off the site, and making a new one. While they hold, a reader gets the manuscript's text only when the check passes on the ledger. The check answers yes or no; it never holds the money.

What the protocol guarantees about its gates

An implementation must:

1. Let the author decrypt their own manuscript at any time.
2. Let the reviewers of a publication decrypt a manuscript offered to it until the publication declines it or the author withdraws it, and refuse them any new decryption after that.
3. Let the renderer decrypt a manuscript only while an agreement covering it is in force, and only the version the publication has approved.
4. Let the renderer build and publish a site when asked by the author of a manuscript the site carries or has carried, a holder, or someone the holders have authorised.
5. Make a manuscript's gates immutable once it is encrypted, so that no one but the implementation's governance, by publishing new code, can change them.

6. Check every gate against the ledger at the moment it is asked, so that an ending or a withdrawal counts from the moment it is recorded.
7. When an agreement includes monetization, release the manuscript’s text to a reader only once the agreed check passes on the ledger.
8. Keep whether, and for how long, a manuscript sits behind the reader gate unchanged for as long as the agreement lasts, so that changing it takes a new agreement.

7. The press

The press turns an author’s signed manuscript into a page a reader can read, and every decryption happens inside it: for the author, for a publication’s reviewers, for building pages and for readers who pass the reader gate. It is the only technical part of the protocol that must hold a manuscript decrypted, so two things about it have to be trusted: the hardware and the people who own it. Everything else it does can be checked. It has three layers: the hardware, the nodes that run on it, and the code the nodes run.

Hardware

The press runs in sealed environments: hardware that lets no one, not even its owner, see into the code as it runs or interfere with it, and that can attest to which code that is. The protocol trusts the hardware to keep that promise, and trusts its owners not to collude against the rights this paper gives authors and publications. Neither trust is ever perfect, but both can be raised to an acceptable degree: for example, by federated governance among a diverse set of node operators, or by collateral freedom: running on a compute network whose operators serve so many other clients that turning against one would cost them too much.

Nodes

Each node runs the press’s code and holds the press’s keys for decryption and signing. The keys can be rotated, and there is a method to recover them. Anyone can read and run the code, but running a copy gets no one past a gate: the keys stay inside the nodes, and the hardware attests which code uses them.

Code

The code is a set of pure functions, each addressed by its content and published by the implementation’s governance. The gates (section 6) are among them, and every decryption must pass one: the decryption that opens a manuscript to its author or to a publication’s reviewers, and the renderer, which builds a publication’s pages.

In a build, the renderer first decrypts the manuscripts whose pages need updating, when the gates pass. It renders them into HTML with the publication’s templates and configuration, which are content-addressed and recorded on the ledger by the publication. Monetized text is encrypted again under the reader gate. For each page, the renderer signs a colophon, a metadata object that carries a hash of the page and points to the manuscript and the agreement; the manuscript carries the author’s address and signature. It builds a Merkle tree of the site’s pages and assets and computes the address of the root. Last, it signs a transaction that updates the ledger record linking the publication’s name to that root.

What the protocol requires of the press

An implementation must:

1. Run the press only in sealed environments that let no one, their owner included, see into the code as it runs or interfere with it, and that attest to which code runs.
2. Choose node operators who are extremely unlikely to collude against the rights this paper gives authors and publications, whether through federated governance among a diverse set of them or through collateral freedom.
3. Keep the press's keys for decryption and signing inside the nodes, with a way to rotate and recover them.
4. Make the gates part of the attested code, and decrypt a manuscript, for anyone, only when a gate passes.
5. Build pages only with pure functions, each addressed by its content and published by the implementation's governance.
6. Render a manuscript's body only if it matches the hash its author signed.
7. Render only with templates and configuration that the publication has recorded on the ledger by their content address.
8. Encrypt monetized text again under the reader gate.
9. Sign a colophon for every page, carrying a hash of the page and pointing to the manuscript and the agreement.
10. Record the root of the site's Merkle tree on the ledger, against the publication's name, in a transaction the press signs.

8. Monetization and costs

The protocol lays three rails for direct payment, and nothing more. First, a destination: every author account can name a wallet, and so can every publication, so whatever a reader pays goes straight there, with no party in between to hold it. Second, a division: the agreement records how any revenue divides, and the ledger carries out the division, so neither party is paid from the other's hand. Neither party can change it alone: changing it means ending the agreement, which takes the manuscript off the site, and making a new one. Finally, a condition: the reader gate (section 6) can ask for anything the ledger can confirm, such as a payment, a membership or a subscription. It answers only yes or no, and never holds the money.

Whether a publication uses these rails is up to its business model, and charging nothing is a business model too. So is how a payment that opens many manuscripts, such as a subscription, is divided among them. Whether a manuscript sits behind the gate, and for how long, is part of the agreement; what the gate asks of a reader is the publication's choice. The protocol neither chooses the model nor needs to know it; what it guarantees is what happens on the rails. How money gets into a reader's wallet before they pay, or out of a wallet after, is up to its owner: the protocol requires no particular route and forbids none.

What the protocol requires of payment

An implementation must:

1. Offer every publication the three rails: wallets for authors and publications, a division recorded in the agreement, and the reader gate.
2. Pay every payment made on the rails straight to the wallets the author and the publication name, with no party in between.
3. Divide it on the ledger as the agreement records it.
4. Keep the division unchanged for as long as the agreement lasts.

Costs

Using the protocol costs something: writing to the ledger, storing manuscripts and pages, and running the press. The costs follow the responsibilities of sections 2 and 4: the author keeps their manuscripts available, and the publication keeps its pages available. An implementation can pay these costs for its users, or leave them with the users, who then pay their own way or through a service of their choice.

9. The reader

A reader can check a page without trusting whoever serves it, by following a chain of authorization from the publication’s name to the page. Each link vouches for the next.

The first link is the name’s record on the ledger. It holds the content address of the site’s root, not the location of a server, and while the publication stays in the protocol, only the press can change it; every change is a public, dated entry.

The second link is the site. Every page in the tree is addressed by its content (section 7). A host can store and serve the site, but cannot change a page without changing its address and the root with it, so an altered page no longer matches what the name points to.

The last link is the page’s colophon, signed by the press. It carries a hash of the page and points to the manuscript, which carries the author’s signature, and to the agreement on the ledger. Together they show whose manuscript the page carries, that author and publication agreed to publish it, and that it has not changed since it was built.

Because the chain starts at the name, it also shows that a page is current. A copy of a page from an older build, even one whose author has since withdrawn it, still carries valid signatures, but the name’s record no longer leads to it.

What the protocol guarantees a reader

An implementation must let any reader:

1. See every change to the name’s record.
2. Check any page against the root the name points to.
3. Check the colophon’s signature.
4. Trace the page to the author’s signature and to the agreement.

Glossary

press

The technical third party between author and publication (section 7): sealed hardware, the nodes that run on it, and the open code they run. Every decryption happens inside it, and it is the only technical part of the protocol that holds a manuscript decrypted. It holds its keys for decryption and signing.

renderer

The function of the press that builds a publication's pages: it decrypts the manuscripts an agreement covers, renders them with the publication's templates, signs each page and records the site's root on the ledger.

manuscript

What the author writes and keeps: everything the author produces, for themselves and for publications, before it is rendered. A manuscript is stored encrypted, and readers never see it directly. It replaces "source material", and "work" wherever "work" means the stored, encrypted thing.

page

What the renderer makes from a manuscript under an agreement: signed HTML, which is what readers see. A publication's site is made of pages.

work

The author's creation in the legal sense, as in "the author's rights over their work". A work covers both the manuscript and the pages made from it. Don't use "work" for the encrypted manuscript or for a rendered page.

press's keys

The keys of the press, for decryption and signing. They stay inside the nodes, can be used only by the attested code and, for decryption, only when a gate passes (section 6); they can be rotated and recovered. How many keys that takes is up to the implementation.

implementation's governance

Whoever an implementation puts in charge of its renderer: it publishes the code the nodes run, the gates included (section 7). It can therefore make the renderer refuse an author, a publication or a manuscript, and it is the only party that can change the rules an agreement runs under, by publishing new code. The author and the publication each have their own governance in the same sense (sections 2 and 4).

collateral freedom

Protection that comes from sharing infrastructure with many others: when the operators of a compute network serve so many other clients that turning against one would cost them too much, that one inherits their reliability. Section 7 offers it as one way to raise the trust placed in the owners of the hardware. The term is borrowed from censorship circumvention, where it names hosting on services too important to block.

colophon

The metadata the renderer signs for every page (section 7): a hash of the page, and pointers to the manuscript and the agreement it was built from; the manuscript carries the author’s address and signature. Named after the printer’s note at the end of a book saying who printed it, when and from which manuscript. Not “imprint”, which stays with the publication’s name and its responsibility.

chain of authorization

The path from a publication’s name to one of its pages, which lets anyone check the page without trusting whoever serves it. It runs from the name, through its record on the ledger, which only the renderer’s signing key can update, and the signed set of pages that record points to, to the proof inside each page. That proof shows that the page’s text is the manuscript as the author signed it, built under an agreement in force. Section 7 introduces the chain; section 9 follows it from the reader’s side. Use “chain of authorization”, not “authorization chain”.